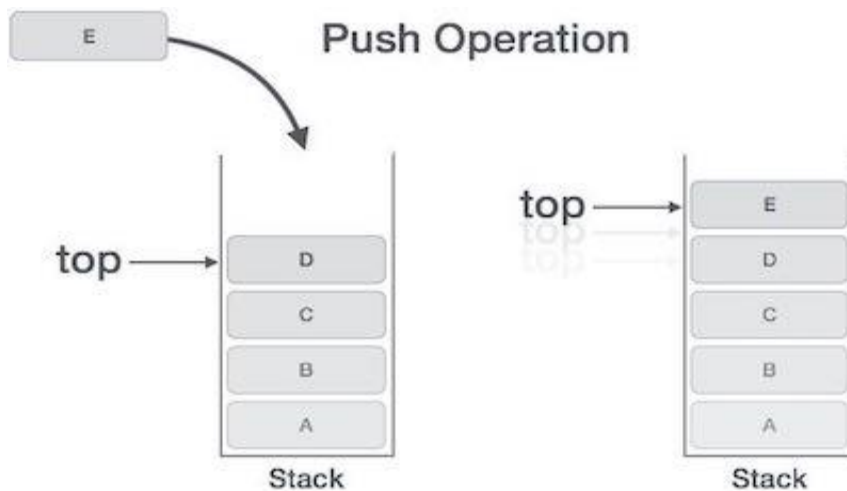


STACKS & QUEUES

- Introduction to Stacks, operations, Applications- Conversion of Infix to postfix, Expression Evaluation
- Introduction to Queues, operations
- Applications-Hot Potato Problem
- Circular Queue and Dequeue

Introduction to Stacks:

Stack is a linear data structure that follows **LIFO** (Last In First Out) Principle, the last element inserted is the first to be popped out. It means both insertion and deletion operations happen at one end only.



LIFO(Last In First Out) Principle:

The LIFO principle means that the last element added to a stack is the first one to be removed.

- New elements are always pushed on top.
- Removal (pop) also happens only from the top.
- This ensures a strict order: last in → first out.

Real-world examples of LIFO:

- Stack of plates – The last plate placed on top is the first one you pick up.
- Shuttlecock box – The last shuttlecock inserted is the first one taken out, since both operations happen from the same end.

Basic Terminologies of Stack:

- **Top:** The position of the most recently inserted element. Insertions (push) and deletions (pop) are always performed at the top.
- **Size:** Refers to the current number of elements present in the stack.

Stack Operations:

There are various stack operations that are applicable on a stack. Stack operations are generally used to extract information and data from a stack data structure.

Some of the stack operations are given below.

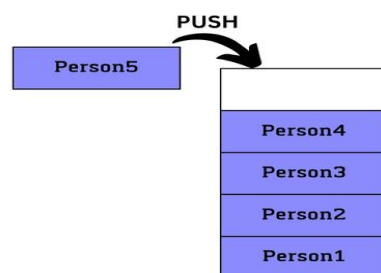
- **push()**
- **pop()**
- **topElement() / peek()**
- **isEmpty()**
- **isFull()**
- **size()**

1. push():

It adds a new element to the top of the stack. It takes the element as its input and adds it to the top of the stack. This operation increases the size of the stack by one. push is a type of function that is basically used to insert an element in the stack, and this is the only method that can insert the element in the stack, which shows how important it is to learn while learning operations on the stack.

Here's the algorithm which will show us how the push() method works:

1. Checks If the stack has space to insert the element
2. If the stack is full, no element can be inserted, so it will return an error
3. Else if the stack has the capacity, the top will be increment by one and will be pointing to the next empty space in the data structure
4. Assign the given element to where the top is pointing in the data structure
5. Inserting is done and shows a success message



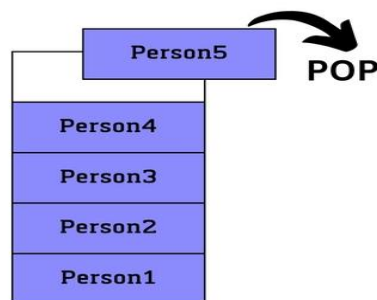
2. pop():

It removes the top element from the stack. It takes no input and removes the top element from the stack. This operation decreases the size of the stack by one. Pop is also a type of method/function that is basically used to remove/delete an element from the

stack, and this is the only method that is used for the removal of an element from the stack.

Here's the algorithm which will show us how the pop() method works:

1. Checks the size of the stack, If the stack is empty. It will return false, as there are no elements to remove
2. If the stack is not empty, the element pointing by the top can be stored in a variable
3. Now 0 can be assigned to the element where the top is pointing
4. The popped value can be returned using the variable



3. topElement() / peek() :

It returns the top element without removing it. This operation does not change the stack size and is useful when inspecting the top element without modifying the stack.

Here's an algorithm of peek() operation for a better understanding:

1. Checks the size of the element, If the stack is empty. An error will be returned as there are no elements to return
2. If the stack is not empty, the element pointing by the top will be stored to a variable
3. Return the variable having the top element

4. isEmpty()

The **isEmpty** Operation is used to check whether the stack is empty. **isEmpty()** returns a **boolean** value (true or false) depending on whether the stack is empty or not.

Here's an algorithm of the isEmpty() operation for a better understanding:

- Checks the top variable, If the top is pointing to -1. Return true, as the top is not pointing to any element
- Else if the top is greater than -1, return false as the top is pointing to some element

5. isFull()

The **isFull** operation is commonly utilized in implementations of a stack using a fixed-size array. This operation determines if the stack is at maximum capacity and cannot hold additional elements. **isfull()** returns a **Boolean** value (true or false) depending on whether the stack is full or not.

Here's an algorithm of the isFull() operation for a better understanding:

- Checks the top variable, If the top is equal to the given size. Return true as the capacity of the data structure is full
- Else if the top is lesser than the given size, return false

6.size():

The size() is used to calculate the size of the stack, which returns the size of the stack. The size is the capacity of the stack or the number of elements a stack can have at a single time. The size() method is used only when the implementation of stacks is done using arrays because there is a limited space while using arrays. The variable 'top' will be used to calculate the size of the stack. For example, If the top is pointing to 3 means there are 4 elements in the stack, so we can return top + 1, resulting in the size of 4 here.

Program for stack operations :

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_SIZE 101
int a[MAX_SIZE];
int top = -1;

void push(int ele){
    if(top<=MAX_SIZE-1){
        a[++top]=ele;
        printf("Pushed: %d\n",ele);
    }
    else{
        printf("Stack is full. Cannot push: %d\n",ele);
    }
}

int pop(){
    if(top>=0){
        int ele = a[top];
        top--;
        printf("Popped: %d\n",ele);
        return ele;
    }
    else{
        printf("Stack is empty. Cannot pop.\n");
        return '-1';
    }
}

int peek(){
    if(top>=0){
        int ele = a[top];
        printf("Peeked: %d\n",ele);
        return ele;
    }
    else{
        printf("Stack is empty. Cannot peek.\n");
        return '-1';
    }
}

int isEmpty(){
    return top==-1;
}

int isFull(){
    return top>=MAX_SIZE;
}
```

```

int main(){
    push(1);
    push(2);
    push(3);

    peek();

    pop();
    pop();
    pop();
    pop();

    peek();
    return 0;
}

```

Conversion of Infix to Postfix and Expression Evaluation:

Expression:

An expression is a combination of:

- Operands → numbers or variables
- Operators – symbols like +, -, * etc

Types of Expressions

1. **Infix Expression**
 - Operator is **between** operands.
 - Example: A + B
2. **Prefix Expression (Polish Notation)**
 - Operator is **before** operands.
 - Example: + A B
3. **Postfix Expression (Reverse Polish Notation)**
 - Operator is **after** operands.
 - Example: A B +

Operator Precedence and Associativity

When converting infix to postfix, we must respect:

- **Precedence:**
^ (highest), * /, + - (lowest)
- **Associativity:**
 - Left to Right: +, -, *, /
 - Right to Left: ^

Conversion of Infix to Postfix:

We use a **stack** to temporarily hold operators.

Algorithm (Step-by-Step)

1. **Scan** the infix expression from **left to right**.
2. **If** the symbol is an **operand**, → directly **add it to the output**.

3. If the symbol is an **operator**, →
 - Pop all operators from stack having **higher or equal precedence**,
 - Then **push** the current operator onto the stack.
4. If the symbol is '(', → **push** it onto the stack.
5. If the symbol is ')', → **pop** operators until '(' is found.
6. **After scanning**, pop all remaining operators from stack to output.

Example:

Symbol scanned	Stack	Postfix
A	Empty	A
^	^	A
B	^	AB
*	*	AB^
C	*	AB^C
/	/	AB^C*
(	/(	AB^C*
D	/(	AB^C*D
*	/(*	AB^C*D
E	/(*	AB^C*DE
-	/(-	AB^C*DE*
F	/(-	AB^C*DE*F
)	/	AB^C*DE*F-
	Empty	AB^C*DE*F-/

Evaluation of Postfix Expression:

Algorithm:

1. Scan expression from left to right.
2. If the symbol is an operand, → push it onto the stack.
3. If the symbol is an operator, →
 - Pop two operands from stack (say b, then a),
 - Apply the operator: a operator b,
 - Push the result back.
4. At the end, the final value in stack is the result.

Example for the evaluation follows below:

Evaluation of Postfix Expression

Postfix $\rightarrow$ a b c * + d - Let, a = 4, b = 3, c = 2, d = 5

Postfix $\rightarrow$ 4 3 2 * + 5 -

Operator/Operand	Action	Stack
4	Push	4
3	Push	4, 3
2	Push	4, 3, 2
*	Pop (2, 3) and $3*2 = 6$ then Push 6	4, 6
+	Pop (6, 4) and $4+6 = 10$ then Push 10	10
5	Push	10, 5
-	Pop (5, 10) and $10-5 = 5$ then Push 5	5



QUEUES

INTRODUCTION TO QUEUES:

Let us explain the concept of queues using the analogies given below.

- People moving on an escalator. The people who got on the escalator first will be the first one to step out of it.
- People waiting for a bus. The first person standing in the line will be the first one to get into the bus.
- People standing outside the ticketing window of a cinema hall. The first person in the line will get the ticket first and thus will be the first one to move out of it.
- Luggage kept on conveyor belts. The bag which was placed first will be the first to come out at the other end.
- Cars lined at a toll bridge. The first car to reach the bridge will be the first to leave.

In all these examples, we see that the element at the first position is served first. Same is the case with queue data structure. A queue is a FIFO (First-In, First-Out) data structure in which the element that is inserted first is the first one to be taken out. The elements in a queue are added at one end called the REAR and removed from the other end called the FRONT. Queues can be implemented by using either arrays or linked lists. In this section, we will see how queues are implemented using each of these data structures.

ARRAY REPRESENTATION & IMPLEMENTATION OF QUEUES

Queues can be easily represented using linear arrays. As stated earlier, every queue has front and rear variables that point to the position from where deletions and insertions can be done, respectively.

The array representation of a queue is shown in Fig. 3.1.

12	9	7	18	14	36				
0	1	2	3	4	5	6	7	8	9

Figure 3.1 Queue

12	9	7	18	14	36	45			
0	1	2	3	4	5	6	7	8	9

Figure 3.2 Queue after insertion of a new element

	9	7	18	14	36	45			
0	1	2	3	4	5	6	7	8	9

Figure 3.2 Queue after deletion of an element

Operations on Queues

In Fig. 3.1, FRONT = 0 and REAR = 5. Suppose we want to add another element with value 45, then REAR would be incremented by 1 and the value would be stored at the position pointed by REAR.

The queue after addition would be as shown in Fig. 3.2. Here, FRONT = 0 and REAR = 6. Every time a new element has to be added, we repeat the same procedure.

If we want to delete an element from the queue, then the value of FRONT will be incremented. Deletions are done from only this end of the queue. The queue after deletion will be as shown in Fig.

3.3.

Here, FRONT = 1 and REAR = 6.

However, before inserting an element in a queue, we must check for overflow conditions. An overflow will occur when we try to insert an element into a queue that is already full. When $\text{REAR} = \text{MAX} - 1$, where MAX is the size of the queue, we have an overflow condition. Note that we have written $\text{MAX} - 1$ because the index starts from 0. Similarly, before deleting an element from a queue, we must check for underflow conditions. An underflow condition occurs when we try to delete an element from a queue that is already empty. If $\text{FRONT} = -1$ and $\text{REAR} = -1$, it means there is no element in the queue.

```
Step 1: IF REAR = MAX-1
        Write OVERFLOW
        Goto step 4
      [END OF IF]
Step 2: IF FRONT = -1 and REAR = -1
        SET FRONT = REAR = 0
      ELSE
        SET REAR = REAR + 1
      [END OF IF]
Step 3: SET QUEUE[REAR] = NUM
Step 4: EXIT
```

Algorithm to insert an element in a queue

```
Step 1: IF FRONT = -1 OR FRONT > REAR
        Write UNDERFLOW
      ELSE
        SET VAL = QUEUE[FRONT]
        SET FRONT = FRONT + 1
      [END OF IF]
Step 2: EXIT
```

Algorithm to delete an element from a queue

NOTE: The process of inserting an element in the queue is called enqueue, and the process of deleting an element from the queue is called dequeue.

LINKED REPRESENTATION & IMPLEMENTATION OF QUEUES

We have seen how a queue is created using an array. Although this technique of creating a queue is easy, its drawback is that the array must be declared to have some fixed size. If we allocate space for 50 elements in the queue and it hardly uses 20–25 locations, then half of the space will be wasted.

And in case we allocate less memory locations for a queue that might end up growing large and large, then a lot of re-allocations will have to be done, thereby creating a lot of overhead and consuming a lot of time.

In case the queue is a very small one or its maximum size is known in advance, then the array implementation of the queue gives an efficient implementation. But if the array size cannot be determined in advance, the other alternative, i.e., the linked representation is used.

The storage requirement of linked representation of a queue with n elements is $O(n)$ and the typical time requirement for operations is $O(1)$.

In a linked queue, every element has two parts, one that stores the data and another that stores the address of the next element. The START pointer of the linked list is used as FRONT. Here, we will also use another pointer called REAR, which will store the address of the last element in the queue.

All insertions will be done at the rear end and all the deletions will be done at the front end. If $\text{FRONT} = \text{REAR} = \text{NULL}$, then it indicates that the queue is empty. The linked representation of a queue is shown in Fig. 3.4.

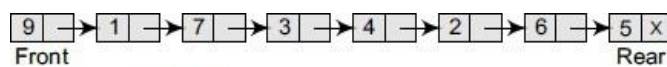


Figure 3.4 Linked queue

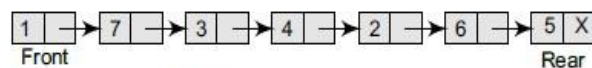


Figure 3.5 Linked queue

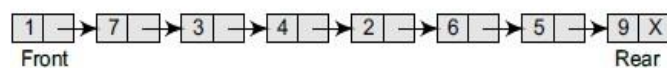


Figure 3.6 Linked queue after inserting a new node

Algorithm to insert an element in a linked queue

```

Step 1: Allocate memory for the new node and name
        it as PTR
Step 2: SET PTR -> DATA = VAL
Step 3: IF FRONT = NULL
        SET FRONT = REAR = PTR
        SET FRONT -> NEXT = REAR -> NEXT = NULL
    ELSE
        SET REAR -> NEXT = PTR
        SET REAR = PTR
        SET REAR -> NEXT = NULL
    [END OF IF]
Step 4: END

```

Insert Operation

The insert operation is used to insert an element into a queue. The new element is added as the last element of the queue. Consider the linked queue shown in Fig. 3.5.

To insert an element with value 9, we first check if $\text{FRONT} = \text{NULL}$. If the condition holds, then the queue is empty. So, we allocate memory for a new node, store the value in its data part and NULL in its next part. The new node will then be called both FRONT and rear. However, if $\text{FRONT} \neq \text{NULL}$, then we will insert the new node at the rear end of the linked queue and name this new node as rear. Thus, the updated queue becomes as shown in Fig. 3.6.

The algorithm shows that inserting an element in a linked queue. In Step 1, the memory is allocated for the new node. In Step 2, the DATA part of the new node is initialized with the value to be stored in the node. In Step 3, we check if the new node is the first node of the linked queue. This is done by checking if $\text{FRONT} = \text{NULL}$. If this is the case, then the new node is tagged as FRONT as well as REAR. Also, NULL is stored in the NEXT part of the node (which is also the FRONT and the REAR node). However, if the new node is not the first node in the list, then it is added at the REAR end of the linked queue (or the last node of the queue).

Delete Operation

The delete operation is used to delete the element that is first inserted in a queue, i.e., the element whose address is stored in FRONT. However, before deleting the value, we must first check if $\text{FRONT} = \text{NULL}$ because if this is the case, then the queue is empty and no more deletions can be

done. If an attempt is made to delete a value from a queue that is already empty, an underflow message is printed. Consider the queue shown in Fig. 3.7.

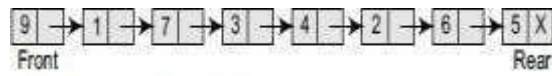


Figure 3.7 Linked queue

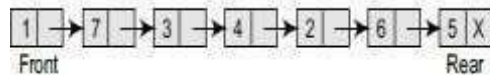


Figure 3.8 Linked queue after deletion of an element

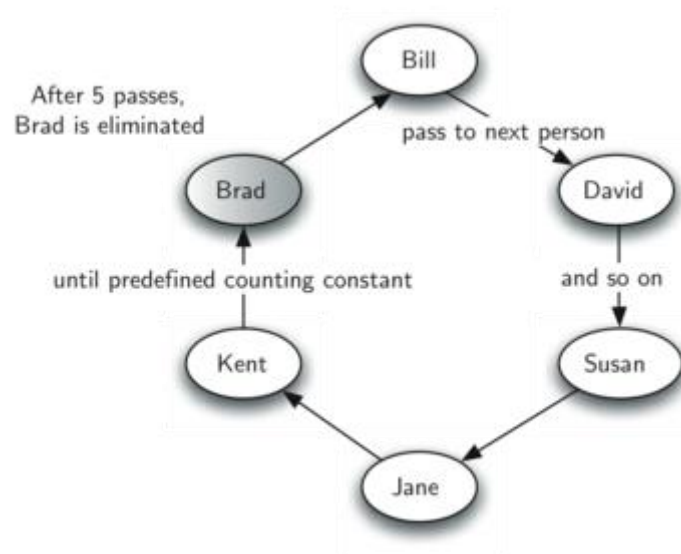
```

Step 1: IF FRONT = NULL
        Write "Underflow"
        Go to Step 5
      [END OF IF]
Step 2: SET PTR = FRONT
Step 3: SET FRONT = FRONT -> NEXT
Step 4: FREE PTR
Step 5: END

```

Applications-Hot Potato Problem

One of the typical applications for showing a queue in action is to simulate a real situation that requires data to be managed in a FIFO manner. To begin, let's consider the children's game hot potato. In this game children line up in a circle and pass an item from neighbor to neighbor as fast as they can. At a certain point in the game, the action is stopped and the child who has the item (the potato) is removed from the circle. Play continues until only one child is left.

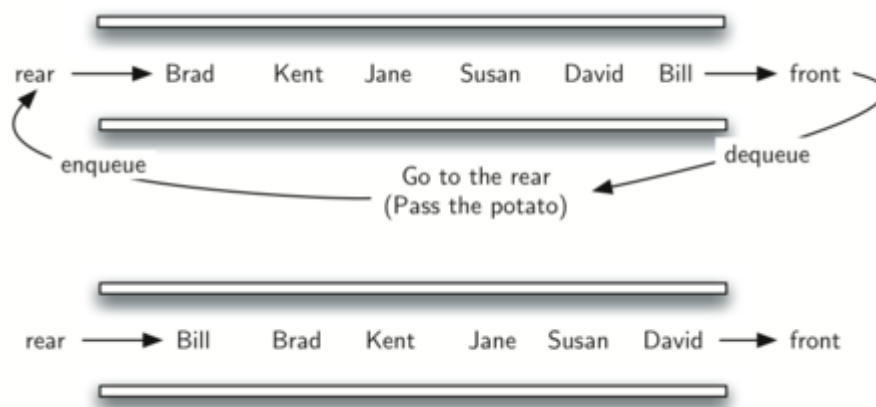


A Six-Person Game of Hot Potato

This game is a modern-day equivalent of the famous Josephus problem. Based on a legend about the famous first-century historian Flavius Josephus, the story is told that in the Jewish revolt against Rome, Josephus and 39 of his comrades held out against the Romans in a cave. With defeat imminent, they decided that they would rather die than be slaves to the Romans. They arranged themselves in a circle. One man was designated as number one, and proceeding clockwise they killed every seventh man. Josephus, according to the legend, was among other things an accomplished mathematician. He instantly figured out where he ought to sit in order to be the last to go. When the time came, instead of killing himself, he joined the Roman side. You can find many different versions of this story. Some count every third man and some allow the last man to escape on a horse. In any case, the idea is the same.

We will implement a general **simulation** of Hot Potato. Our program will input a list of names and a value, call it num to be used for counting how many times the potato is passed before the action is stopped. The method will return the name of the last person remaining after repetitive counting by num. What happens at that point is up to you.

To simulate the circle, we will use a queue. Assume that the child holding the potato will be at the head of the queue. Upon passing the potato, the simulation will dequeue and then immediately enqueue that child, putting them at the end of the line. They will then wait until all the others have been at the head before it will be their turn again. After num dequeue/enqueue operations, the child at the front will be removed permanently and another cycle will begin. This process will continue until only one name remains (the size of the queue is 1).



A Queue Implementation of Hot Potato

Circular Queues

In linear queues, we have discussed so far that insertions can be done only at one end called the REAR and deletions are always done from the other end called the FRONT. Look at the queue shown in Fig. 3.9.

54	9	7	18	14	36	45	21	99	72
0	1	2	3	4	5	6	7	8	9

Figure 3.9 Linear queue

Here, FRONT = 0 and REAR = 9.

Now, if you want to insert another value, it will not be possible because the queue is completely full. There is no empty space where the value can be inserted. Consider a scenario in which two successive deletions are made. The queue will then be given as shown in Fig. 3.10.

		7	18	14	36	45	21	99	72
0	1	2	3	4	5	6	7	8	9

Figure 3.10 Queue after two successive deletions

Here, front = 2 and REAR = 9.

Suppose we want to insert a new element in the queue shown in Fig. 3.10. Even though there is space available, the overflow condition still exists because the condition $\text{rear} = \text{MAX} - 1$ still holds true. This is a major drawback of a linear queue.

To resolve this problem, we have two solutions. First, shift the elements to the left so that the vacant space can be occupied and utilized efficiently. But this can be very time-consuming, especially when the queue is quite large.

The second option is to use a circular queue. In the circular queue, the first index comes right after the last index. Conceptually, you can think of a circular queue as shown in Fig. 3.11.

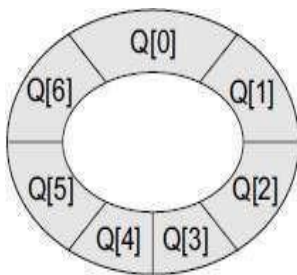


Figure 3.11 Circular queue

The circular queue will be full only when $\text{front} = 0$ and $\text{rear} = \text{Max} - 1$. A circular queue is implemented in the same manner as a linear queue is implemented. The only difference will be in the code that performs insertion and deletion operations.

For insertion, we now have to check for the following three conditions:

- If $\text{front} = 0$ and $\text{rear} = \text{MAX} - 1$, then the circular queue is full. Look at the queue given in Fig. 3.12 which illustrates this point.
- If $\text{rear} \neq \text{MAX} - 1$, then rear will be incremented and the value will be inserted as illustrated in Fig. 3.13.
- If $\text{front} \neq 0$ and $\text{rear} = \text{MAX} - 1$, then it means that the queue is not full. So, set $\text{rear} = 0$ and insert the new element there, as shown in Fig. 3.14.

90	49	7	18	14	36	45	21	99	72
0	1	2	3	4	5	6	7	8	9

FRONT = 0 REAR = 9

Figure 3.12 Full queue

90	49	7	18	14	36	45	21	99	
0	1	2	3	4	5	6	7	8	9

FRONT = 0 REAR = 8

Increment rear so that it points to location 9 and insert the value here

Figure 3.13 Queue with vacant locations

		7	18	14	36	45	21	80	81
0	1	2	3	4	5	6	7	8	9

Set REAR = 0 and insert the value here

Figure 3.14 Inserting an element in a circular queue

```

Step 1: IF FRONT = 0 and REAR = MAX - 1
        Write "OVERFLOW"
        Goto step 4
      [End OF IF]
Step 2: IF FRONT = -1 and REAR = -1
        SET FRONT = REAR = 0
      ELSE IF REAR = MAX - 1 and FRONT != 0
        SET REAR = 0
      ELSE
        SET REAR = REAR + 1
      [END OF IF]
Step 3: SET QUEUE[REAR] = VAL
Step 4: EXIT

```

Let us look at the algorithm to insert an element in a circular queue. In Step 1, we check for the overflow condition. In Step 2, we make two checks. First to see if the queue is empty, and second to see if the REAR end has already reached the maximum capacity while there are certain free locations before the FRONT end. In Step 3, the value is stored in the queue at the location pointed by REAR.

Let us now discuss how **deletions** are performed in this case. To delete an element, again we check for three conditions.

- Look at Fig. 3.15. If front = -1, then there are no elements in the queue. So, an underflow condition will be reported.
- If the queue is not empty and front = rear, then after deleting the element at the front the queue becomes empty and so front and rear are set to -1. This is illustrated in Fig. 3.16.
- If the queue is not empty and front = MAX-1, then after deleting the element at the front, front is set to 0. This is shown in Fig. 3.17.

0	1	2	3	4	5	6	7	8	9

FRONT = REAR = -1

Figure 3.15 Empty queue

									81
0	1	2	3	4	5	6	7	8	9

FRONT = REAR = 9

Delete this element and set REAR = FRONT = -1

Figure 3.16 Queue with a single element

72	63	9	18	27	39				81
0	1	2	3	4	5	6	7	8	9

rear = 5 FRONT = 9

Delete this element and set FRONT = 0

Figure 3.17 Queue where FRONT = MAX-1 before deletion

```

Step 1: IF FRONT = -1
        Write "UNDERFLOW"
        Goto Step 4
      [END of IF]
Step 2: SET VAL = QUEUE[FRONT]
Step 3: IF FRONT = REAR
        SET FRONT = REAR = -1
      ELSE
        IF FRONT = MAX - 1
          SET FRONT = 0
        ELSE
          SET FRONT = FRONT + 1
        [END of IF]
      [END OF IF]
Step 4: EXIT

```


Dequeues

A deque (pronounced as ‘deck’ or ‘dequeue’) is a list in which the elements can be inserted or deleted at either end. It is also known as a head-tail linked list because elements can be added to or removed from either the front (head) or the back (tail) end.

However, no element can be added and deleted from the middle. In the computer’s memory, a deque is implemented using either a circular array or a circular doubly linked list.

In a deque, two pointers are maintained, LEFT and RIGHT, which point to either end of the deque. The elements in a deque extend from the LEFT end to the RIGHT end and since it is circular, Dequeue[N-1] is followed by Dequeue [0]. Consider the deques shown in Fig. 3.18.

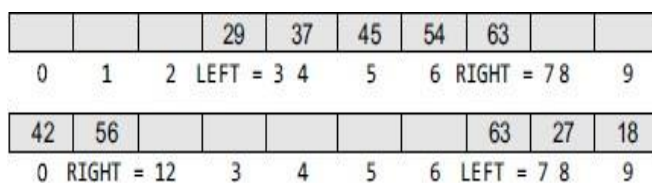


Figure 3.18 Double-ended queues

There are two variants of a double-ended queue. They include

- *Input restricted deque* In this deque, insertions can be done only at one of the ends, while deletions can be done from both ends.
- *Output restricted deque* In this deque, deletions can be done only at one of the ends, while insertions can be done on both ends.

Program:

```
#include<stdio.h>
#include<stdlib.h> #define MAX 5
int dq[MAX];
int front = -1, rear = -1;

//Insertion from front void insert_front(int x)
{
    if(front == (rear+1)%MAX)
    {
        printf("Deque is overflow!!!\n");
        return;    } if(front == -1)    front=rear=0;    else if (front==0)
        front = MAX-1;    else
        front--;    dq[front]=x;
    }

// insertion from rear end void insert_rear(int x)
```

```

{
    if(front == (rear+1)%MAX)
    {
        printf("Deque is overflow!!!\n");
        return;
    }

    if(front==-1)
    {
        rear = 0;    front = 0;
    }
    else
        rear = (rear+1)%MAX;
    dq[rear]=x;
}

void delete_front()
{
    if(front==-1)
    {
        printf("Deque is underflow!!!\n");
        return;
    }
    printf("Deleted element from the front: %d\n", dq[front]);    if(front==rear)
    {
        rear=-1;    front=-1;
    }
    else
        front = (front+1)%MAX;
}

// dequeue_rear() function deletes the element from the rear void delete_rear()
{
    if(front==-1)
    {
        printf("Deque is underflow!!!\n");
        return;
    }
    printf("Deleted element from rear: %d\n", dq[rear]);    if(front==rear)
    {
        front=-1;    rear=-1;
    }
    else if(rear == 0)
        rear = MAX-1;    else
        rear = rear-1;
}

//function to analyze state of queue.
void display()

```



```

{   int i=front;
    if(front==-1)
    {
        return;
    }
    printf("\nDeque Elements: ");    for(i=front;i!=rear;i=(i+1)%MAX)        printf("%d ",dq[i]);
    printf("%d\n",dq[rear]);
}

int main()
{
    int op, x;        while(1)
    {

        printf("1.Insert_Front\n2.Insert_Rear\n3.Delete_Front\n4.Delete_Rear\n5.Display\n6.Exit\n");
        printf("Option: ");        scanf("%d",&op);
        switch(op) {
            case 1:
                printf("element: ");
                scanf("%d",&x);
                insert_front(x);
                break;
            case 2:
                printf("element: ");
                scanf("%d",&x);
                insert_rear(x);
                break;
            case 3:
                delete_front();
                break;
            case 4:
                delete_rear();
                break;
            case 5:
                display();
                break;
            case 6: exit(0);
        }
    }
    return 0; }

```